

1.

Input File	k	m	FIFO	LRU	OPTFF
File 1	10	50	24	32	16
File 2	12	63	24	18	15
File 3	10	54	34	41	17

- Yes, OPTFF has the fewest misses. This is because OPTFF will always evict the item whose next use is the farthest into the future. It knows the most optimal entire future request sequence, allowing it to make the most optimal decision when choosing which item to evict.
- FIFO generally has less cache misses than LRU except for when the cache capacity increases to 12. In that case, LRU has less cache misses than FIFO; this is because FIFO performs worse once cache size is increased, this is known as Belady's Anomaly.

2. There does exist a request sequence for which OPTFF incurs strictly fewer misses than LRU or FIFO.

Here is a constructed sequence where $k=3$

1 2 3 4 1 2 3 4 1 2 3 4

- LRU misses all 12 requests because LRU evicts the least recently used item on every miss. With this sequence every request to item 4 will evict whichever item was loaded first, this cycle of misses will repeat.
 - OPTFF is able to see the full sequence so it knows that item 4 will not be needed as soon as 1, 2, or 3 will be. So it keeps 1, 2, and 3 in cache after the first four misses, which evicts 4 every time it should be inserted. But 1, 2, and 3 are hits.
 - So, this results with LRU having 12 misses and OPTFF having 4 misses. OPTFF is able to recognize that 1, 2, and 3 will be reused sooner than 4 making it incur strictly fewer misses than LRU or FIFO.
3. The number of misses of OPTFF is no larger than that of (A), any offline algorithm that knows the full request sequence, on any fixed sequence due to an exchange argument.
- We are going to assume that both algorithms behave identically until there is some request where the cache is full and an eviction must take place, we are going to consider the first step where OPTFF and (A) disagree.
 - Let: OPTFF evict item X and (A) evict item Y which is not equal to X
 - Item Y will be requested no later than item X because item X is the item whose next request will occur the farthest in the future out of all items currently in the cache due to the definition of OPTFF since it is able to see the full sequence.
 - The next step would be to construct a modified (A) algorithm that agrees with OPTFF where (A) will also evict X, not Y. So now (A) will instead hold Y in cache and evict X.
 - Evicting X cannot cause additional misses if it is never used again.

- Evicting X will delay the next cache miss more than evicting Y if and only if the next request for X is used later than the next request for Y.
- This shows that making (A) agree with OPTFF's decision will not create additional misses. So, (A) can be transformed into an OPTFF algorithm without creating additional misses, proving OPTFF is optimal and that the number of misses of OPTFF is no larger than that of (A) on any fixed sequence.